

OpenCV-Python 기초 사용법

1) 영상의 속성과 픽셀 값 참조

영상의 속성과 픽셀 값 참조

- OpenCV는 영상 데이터를 **numpy.ndarray**로 표현

```
import cv2
```

```
img1 = cv2.imread('cat.bmp', cv2.IMREAD_GRAYSCALE)
```

```
img2 = cv2.imread('cat.bmp', cv2.IMREAD_COLOR)
```

numpy.ndarray

- ndim: 차원 수. len(img.shape)과 같음.
- shape: 각 차원의 크기. (h, w) 또는 (h, w, 3)
 - ↑ 그레이스케일 영상
 - ↑ 컬러 영상
- size: 전체 원소 개수
- dtype: 원소의 데이터 타입. 영상 데이터는 **uint8**.

영상의 속성과 픽셀 값 참조

■ OpenCV 영상 데이터 자료형과 NumPy 자료형

OpenCV 자료형 (1채널)	NumPy 자료형	구분
cv2.CV_8U	numpy.uint8	8비트 부호없는 정수
cv2.CV_8S	numpy.int8	8비트 부호있는 정수
cv2.CV_16U	numpy.uint16	16비트 부호없는 정수
cv2.CV_16S	numpy.int16	16비트 부호있는 정수
cv2.CV_32S	numpy.int32	32비트 부호있는 정수
cv2.CV_32F	numpy.float32	32비트 부동소수형
cv2.CV_64F	numpy.float64	64비트 부동소수형
cv2.CV_16F	numpy.float16	16비트 부동소수형

- 그레이스케일 영상: `cv2.CV_8UC1` → `numpy.uint8, shape = (h, w)`
- 컬러 영상: `cv2.CV_8UC3` → `numpy.uint8, shape = (h, w, 3)`

영상의 속성과 픽셀 값 참조

■ 영상의 속성 참조 예제

```
img1 = cv2.imread('cat.bmp', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('cat.bmp', cv2.IMREAD_COLOR)

print('type(img1):', type(img1)) # type(img1): <class 'numpy.ndarray'>
print('img1.shape:', img1.shape) # img1.shape: (480, 640)
print('img2.shape:', img2.shape) # img2.shape: (480, 640, 3)
print('img2.dtype:', img2.dtype) # img2.dtype: uint8

h, w = img2.shape[:2] # h: 480, w: 640
print('img2 size: {} x {}'.format(w, h))

if len(img1.shape) == 2:
    print('img1 is a grayscale image')
elif len(img1.shape) == 3:
    print('img1 is a truecolor image')

cv2.imshow("img1", img1)
cv2.imshow("img2", img2)
cv2.waitKey()

cv2.destroyAllWindows()
```

영상의 속성과 픽셀 값 참조

■ 영상의 픽셀 값 참조 예제

```
img1 = cv2.imread('cat.bmp', cv2.IMREAD_GRAYSCALE)
img2 = cv2.imread('cat.bmp', cv2.IMREAD_COLOR)
```

```
for y in range(h):
    for x in range(w):
        img1[y, x] = 255
        img2[y, x] = [0, 0, 255]
```

} for문으로 픽셀 값을 변경하는 작업은 매우 느리므로,
픽셀 값 참조 방법만 확인하고 실제로는 사용 금지

```
# img1[:, :] = 255
# img2[:, :] = (0, 0, 255)
```

```
cv2.imshow("img1", img1)
cv2.imshow("img2", img2)
cv2.waitKey()
```

```
cv2.destroyAllWindows()
```

OpenCV-Python 기초 사용법

2) 영상의 생성, 복사, 부분 영상 추출

영상의 생성, 복사, 부분 영상 추출

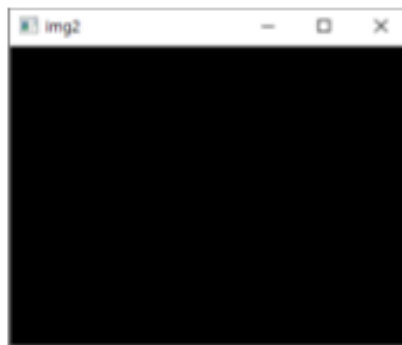
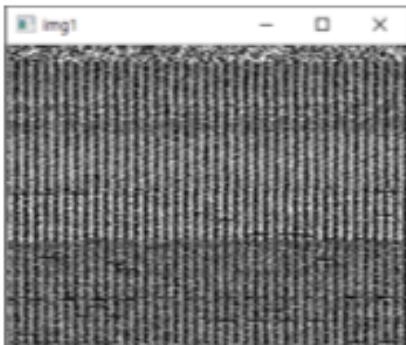
■ 지정한 크기로 새 영상 생성하기

```
numpy.empty(shape, dtype=float, ...) -> arr  
numpy.zeros(shape, dtype=float, ...) -> arr  
numpy.ones(shape, dtype=None, ...) -> arr  
numpy.full(shape, fill_value, dtype=None, ...) -> arr
```

- shape: 각 차원의 크기. (h, w) 또는 (h, w, 3)
- dtype: 원소의 데이터 타입. 일반적인 영상이면 `numpy.uint8` 지정
- arr: 생성된 영상(`numpy.ndarray`)
- 참고사항:
 - `numpy.empty()` 함수는 임의의 값으로 초기화된 배열을 생성
 - `numpy.zeros()` 함수는 0으로 초기화된 배열을 생성
 - `numpy.ones()` 함수는 1로 초기화된 배열을 생성
 - `numpy.full()` 함수는 `fill_value`로 초기화된 배열을 생성

영상의 생성 예제 코드

```
img1 = np.empty((480, 640), dtype=np.uint8)      # grayscale image
img2 = np.zeros((480, 640, 3), dtype=np.uint8)    # color image
img3 = np.ones((480, 640), dtype=np.uint8) * 255 # white
img4 = np.full((480, 640, 3), (0, 255, 255), dtype=np.uint8) # yellow
```

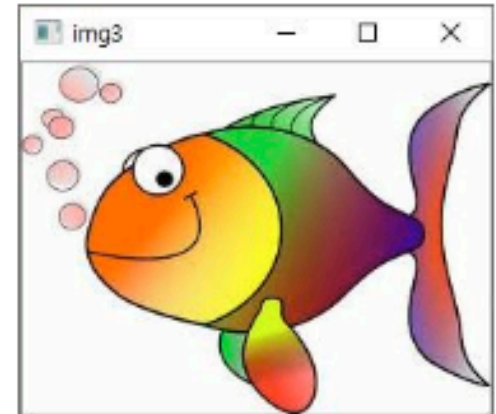
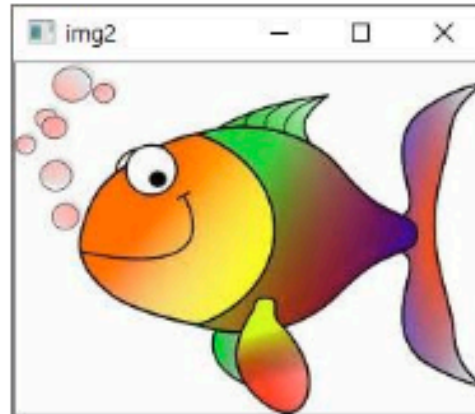
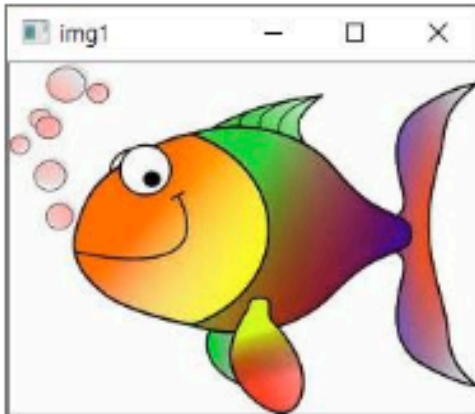


```
cv2.imshow('img1', img1)
cv2.imshow('img2', img2)
cv2.imshow('img3', img3)
cv2.imshow('img4', img4)
cv2.waitKey()
cv2.destroyAllWindows()
```


영상의 생성, 복사, 부분 영상 추출

■ 영상의 참조 및 복사 예제 코드

```
img1 = cv2.imread('HappyFish.jpg')  
  
img2 = img1  
img3 = img1.copy()
```

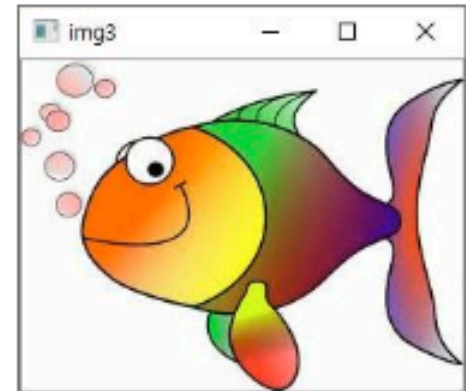
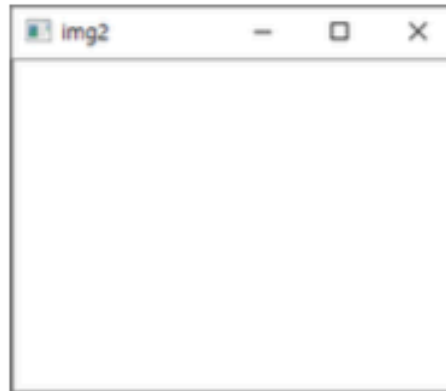
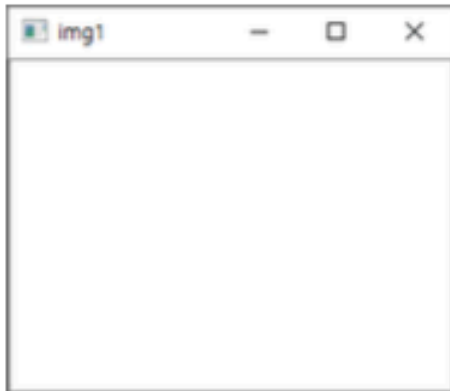


```
cv2.imshow('img1', img1)  
cv2.imshow('img2', img2)  
cv2.imshow('img3', img3)  
cv2.waitKey()  
cv2.destroyAllWindows()
```

영상의 생성, 복사, 부분 영상 추출

■ 영상의 참조 및 복사 예제 코드

```
img1 = cv2.imread('HappyFish.jpg')  
  
img2 = img1  
img3 = img1.copy()  
  
img1.fill(255)
```

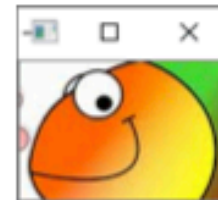
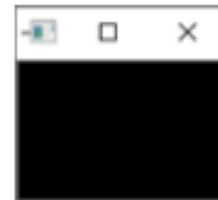


```
cv2.imshow('img1', img1)  
cv2.imshow('img2', img2)  
cv2.imshow('img3', img3)  
cv2.waitKey()  
cv2.destroyAllWindows()
```

영상의 생성, 복사, 부분 영상 추출

■ 부분 영상 추출

```
img1 = cv2.imread('HappyFish.jpg')  
  
img2 = img1[40:120, 30:150] # numpy.ndarray의 슬라이싱  
img3 = img1[40:120, 30:150].copy()  
  
img2.fill(0)
```



```
cv2.imshow('img1', img1)  
cv2.imshow('img2', img2)  
cv2.imshow('img3', img3)  
cv2.waitKey()  
cv2.destroyAllWindows()
```

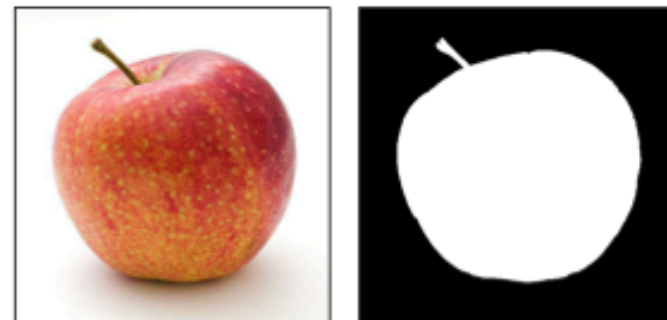
OpenCV-Python 기초 사용법

3) 마스크 연산과 ROI

마스크 연산과 ROI

■ ROI

- Region of Interest, 관심 영역
- 영상에서 특정 연산을 수행하고자 하는 임의의 부분 영역



■ 마스크 연산

- OpenCV는 일부 함수에 대해 ROI 연산을 지원하며, 이때 **마스크 영상**을 인자로 함께 전달해야 함
(e.g.) `cv2.copyTo()`, `cv2.calcHist()`, `cv2.bitwise_or()`, `cv2.matchTemplate()`, etc.
- 마스크 영상은 `cv2.CV_8UC1` 타입(그레이스케일 영상)
- 마스크 영상의 픽셀 값이 0이 아닌 위치에서만 연산이 수행됨
→ 보통 마스크 영상으로는 0 또는 255로 구성된 이진 영상(binary image)을 사용

마스크 연산과 ROI

■ 마스크 연산을 지원하는 픽셀 값 복사 함수

```
cv2.copyTo(src, mask, dst=None) -> dst
```

- src: 입력 영상
- mask: 마스크 영상. cv2.CV_8U. (numpy.uint8)
0이 아닌 픽셀에 대해서만 복사 연산을 수행.
- dst: 출력 영상. 만약 src와 크기 및 타입이 같은 dst를 입력으로 지정하면 dst를 새로 생성하지 않고 연산을 수행.
그렇지않으면 dst를 새로 생성하여 연산을 수행한 후 반환함.

마스크 연산과 ROI

■ 마스크 연산 예제

```
src = cv2.imread('airplane.bmp', cv2.IMREAD_COLOR)
mask = cv2.imread('mask_plane.bmp', cv2.IMREAD_GRAYSCALE)
dst = cv2.imread('field.bmp', cv2.IMREAD_COLOR)
```

```
cv2.copyTo(src, mask, dst)
```

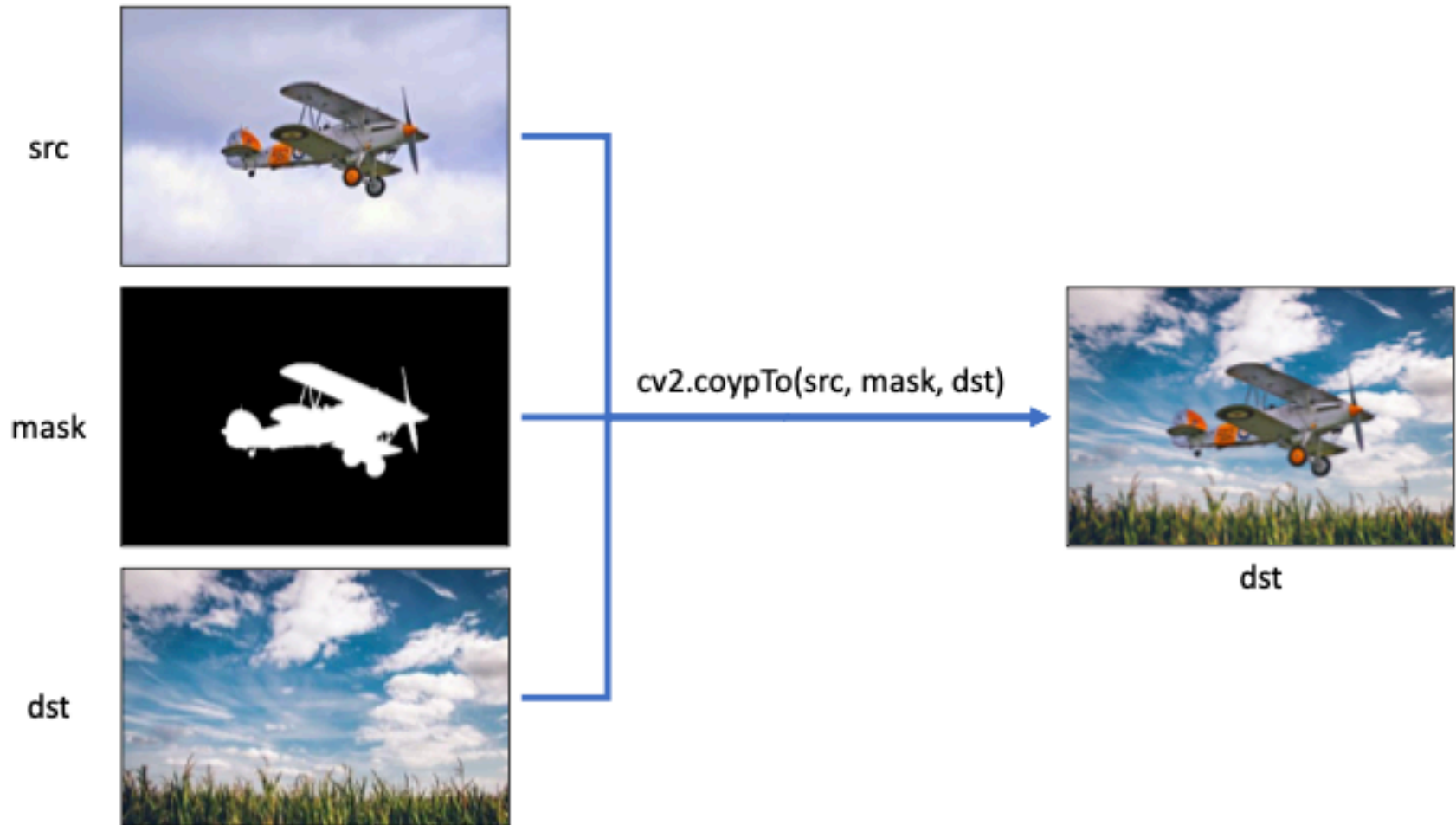
src, mask, dst는 모두 크기가 같아야 함.

src와 dst는 같은 타입이어야 하고, mask는 그레이스케일 타입의 이진 영상.

- NumPy의 불리언 인덱싱(Boolean indexing)을 이용한 마스크 연산

```
dst[mask > 0] = src[mask > 0]
```

마스크 연산과 ROI



마스크 연산과 ROI

```
import sys
import cv2

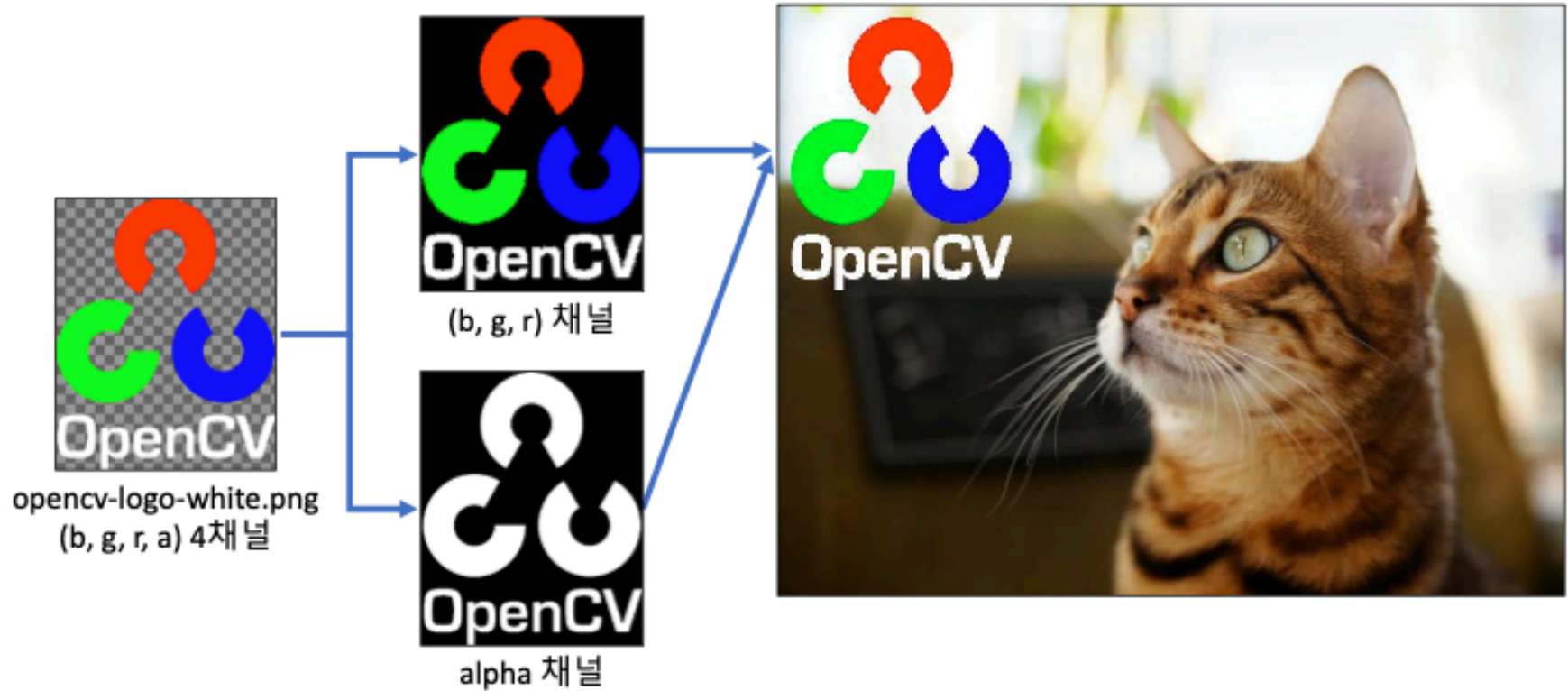
# 마스크 영상을 이용한 영상 합성
src = cv2.imread('airplane.bmp', cv2.IMREAD_COLOR)
mask = cv2.imread('mask_plane.bmp', cv2.IMREAD_GRAYSCALE)
dst = cv2.imread('field.bmp', cv2.IMREAD_COLOR)

if src is None or mask is None or dst is None:
    print('Image load failed!')
    sys.exit()

cv2.copyTo(src, mask, dst)
# dst[mask > 0] = src[mask > 0]

cv2.imshow('src', src)
cv2.imshow('dst', dst)
cv2.imshow('mask', mask)
cv2.waitKey()
cv2.destroyAllWindows()
```

마스크 연산과 ROI



마스크 연산과 ROI

```
import sys
import cv2

# # 알파 채널을 마스크 영상으로 이용
src = cv2.imread("cat.bmp", cv2.IMREAD_COLOR)
logo = cv2.imread("opencv-logo-white.png", cv2.IMREAD_UNCHANGED)

if src is None or logo is None:
    print("Image load failed!")
    sys.exit()

mask = logo[:, :, 3] # mask는 알파 채널로 만든 마스크 영상
logo = logo[:, :, :-1] # logo는 b, g, r 3채널로 구성된 컬러 영상
h, w = mask.shape[:2]
crop = src[10 : 10 + h, 10 : 10 + w] # logo, mask와 같은 크기의 부분 영상 추출

cv2.copyTo(logo, mask, crop)
# crop[mask > 0] = logo[mask > 0]

cv2.imshow("src", src)
cv2.imshow("logo", logo)
cv2.imshow("mask", mask)
cv2.waitKey()
cv2.destroyAllWindows()
```

OpenCV-Python 기초 사용법

4) OpenCV 그리기 함수

OpenCV 그리기 함수

■ OpenCV 그리기 함수

- OpenCV는 영상에 선, 도형, 문자열을 출력하는 그리기 함수를 제공
- 선 그리기: 직선, 화살표, 마커 등
- 도형 그리기: 사각형, 원, 타원, 다각형 등
- 문자열 출력

■ 그리기 함수 사용 시 주의할 점

- 그리기 알고리즘을 이용하여 영상의 픽셀 값 자체를 변경
→ 원본 영상이 필요하면 복사본을 만들어서 그리기 & 출력
- 그레이스케일 영상에는 컬러로 그리기 안 됨
→ `cv2.cvtColor()` 함수로 BGR 컬러 영상으로 변환한 후 그리기 함수 호출

OpenCV 그리기 함수

■ 직선 그리기

```
cv2.line(img, pt1, pt2, color, thickness=None, lineType=None,  
         shift=None) -> img
```

- img: 그림을 그릴 영상
- pt1, pt2: 직선의 시작점과 끝점. (x, y) 튜플.
- color: 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.
- thickness: 선 두께. 기본값은 1.
- lineType: 선 타입. cv2.LINE_4, cv2.LINE_8, cv2.LINE_AA 중 선택.
기본값은 cv2.LINE_8
- shift: 그리기 좌표 값의 축소 비율. 기본값은 0.

OpenCV 그리기 함수

■ 사각형 그리기

```
cv2.rectangle(img, pt1, pt2, color, thickness=None, lineType=None,
               shift=None) -> img
cv2.rectangle(img, rec, color, thickness=None, lineType=None,
               shift=None) -> img
```

- img: 그림을 그릴 영상
- pt1, pt2: 사각형의 두 꼭지점 좌표. (x, y) 튜플.
- rec: 사각형 위치 정보. (x, y, w, h) 튜플.
- color: 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.
- thickness: 선 두께. 기본값은 1. 음수(-1)를 지정하면 내부를 채움.
- lineType: 선 타입. cv2.LINE_4, cv2.LINE_8, cv2.LINE_AA 중 선택. 기본값은 cv2.LINE_8
- shift: 그리기 좌표 값의 축소 비율. 기본값은 0.

OpenCV 그리기 함수

■ 원 그리기

```
cv2.circle(img, center, radius, color, thickness=None, lineType=None,  
           shift=None) -> img
```

- img: 그림을 그릴 영상
- center: 원의 중심 좌표. (x, y) 튜플.
- radius: 원의 반지름
- color: 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.
- thickness: 선 두께. 기본값은 1. 음수(-1)를 지정하면 내부를 채움.
- lineType: 선 타입. cv2.LINE_4, cv2.LINE_8, cv2.LINE_AA 중 선택.
기본값은 cv2.LINE_8
- shift: 그리기 좌표 값의 축소 비율. 기본값은 0.

OpenCV 그리기 함수

■ 다각형 그리기

```
cv2.polylines(img, pts, isClosed, color, thickness=None, lineType=None, shift=None) -> img
```

- img: 그림을 그릴 영상
- pts: 다각형 외곽 점들의 좌표 배열. [numpy.ndarray의 리스트](#).
(e.g.) `[np.array([[10, 10], [50, 50], [10, 50]], dtype=np.int32)]`
- isClosed: 폐곡선 여부. True 또는 False 지정.
- color: 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.
- thickness: 선 두께. 기본값은 1. 음수(-1)를 지정하면 내부를 채움.
- lineType: 선 타입. `cv2.LINE_4`, `cv2.LINE_8`, `cv2.LINE_AA` 중 선택.
기본값은 `cv2.LINE_8`
- shift: 그리기 좌표 값의 축소 비율. 기본값은 0.

OpenCV 그리기 함수

■ 문자열 출력

```
cv2.putText(img, text, org, fontFace, fontScale, color, thickness=None,  
            lineType=None, bottomLeftOrigin=None) -> img
```

- img: 그림을 그릴 영상
- text: 출력할 문자열
- org: 영상에서 문자열을 출력할 위치의 좌측 하단 좌표. (x, y) 튜플.
- fontFace: 폰트 종류. `cv2.FONT_HERSHEY_` 로 시작하는 상수 중 선택
- fontScale: 폰트 크기 확대/축소 비율
- color: 선 색상 또는 밝기. (B, G, R) 튜플 또는 정수값.
- thickness: 선 두께. 기본값은 1. 음수(-1)를 지정하면 내부를 채움.
- lineType: 선 타입. `cv2.LINE_4`, `cv2.LINE_8`, `cv2.LINE_AA` 중 선택.
- bottomLeftOrigin: True이면 영상의 좌측 하단을 원점으로 간주. 기본값은 False.

OpenCV 그리기 함수

- cv2.putText()에서 지원하는 fontFace 상수와 실제 출력 모양

FONT_HERSHEY_SIMPLEX

FONT_HERSHEY_PLAIN

FONT_HERSHEY_DUPLEX

FONT_HERSHEY_COMPLEX

FONT_HERSHEY_TRIPLEX

FONT_HERSHEY_COMPLEX_SMALL

FONT_HERSHEY_SCRIPT_SIMPLEX

FONT_HERSHEY_SCRIPT_COMPLEX

FONT_HERSHEY_COMPLEX | FONT_ITALIC

OpenCV 그리기 함수

■ 다양한 그리기 함수 실행 예제

```
img = np.full((400, 400, 3), 255, np.uint8)

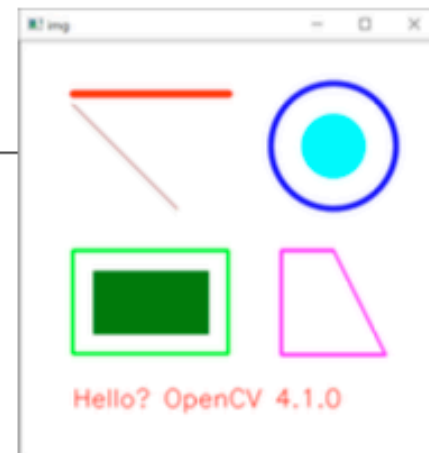
cv2.line(img, (50, 50), (200, 50), (0, 0, 255), 5)
cv2.line(img, (50, 60), (150, 160), (0, 0, 128))

cv2.rectangle(img, (50, 200, 150, 100), (0, 255, 0), 2)
cv2.rectangle(img, (70, 220), (180, 280), (0, 128, 0), -1)

cv2.circle(img, (300, 100), 60, (255, 0, 0), 3, cv2.LINE_AA)
cv2.circle(img, (300, 100), 30, (255, 255, 0), -1, cv2.LINE_AA)

pts = np.array([[250, 200], [300, 200], [350, 300], [250, 300]])
cv2.polylines(img, [pts], True, (255, 0, 255), 2)

text = 'Hello? OpenCV ' + cv2.__version__
cv2.putText(img, text, (50, 350), cv2.FONT_HERSHEY_SIMPLEX, 0.8,
            (0, 0, 255), 1, cv2.LINE_AA)
```



OpenCV 그리기 함수

```
import numpy as np
import cv2

img = np.full((400, 400, 3), 255, np.uint8)

cv2.line(img, (50, 50), (200, 50), (0, 0, 255), 5)
cv2.line(img, (50, 60), (150, 160), (0, 0, 128))

cv2.rectangle(img, (50, 200, 150, 100), (0, 255, 0), 2)
cv2.rectangle(img, (70, 220), (180, 280), (0, 128, 0), -1)

cv2.circle(img, (300, 100), 30, (255, 255, 0), -1, cv2.LINE_AA)
cv2.circle(img, (300, 100), 60, (255, 0, 0), 3, cv2.LINE_AA)

pts = np.array([[250, 200], [300, 200], [350, 300], [250, 300]])
cv2.polylines(img, [pts], True, (255, 0, 255), 2)

text = 'Hello? OpenCV ' + cv2.__version__
cv2.putText(img, text, (50, 350), cv2.FONT_HERSHEY_SIMPLEX, 0.8,
            (0, 0, 255), 1, cv2.LINE_AA)

cv2.imshow("img", img)
cv2.waitKey()
cv2.destroyAllWindows()
```

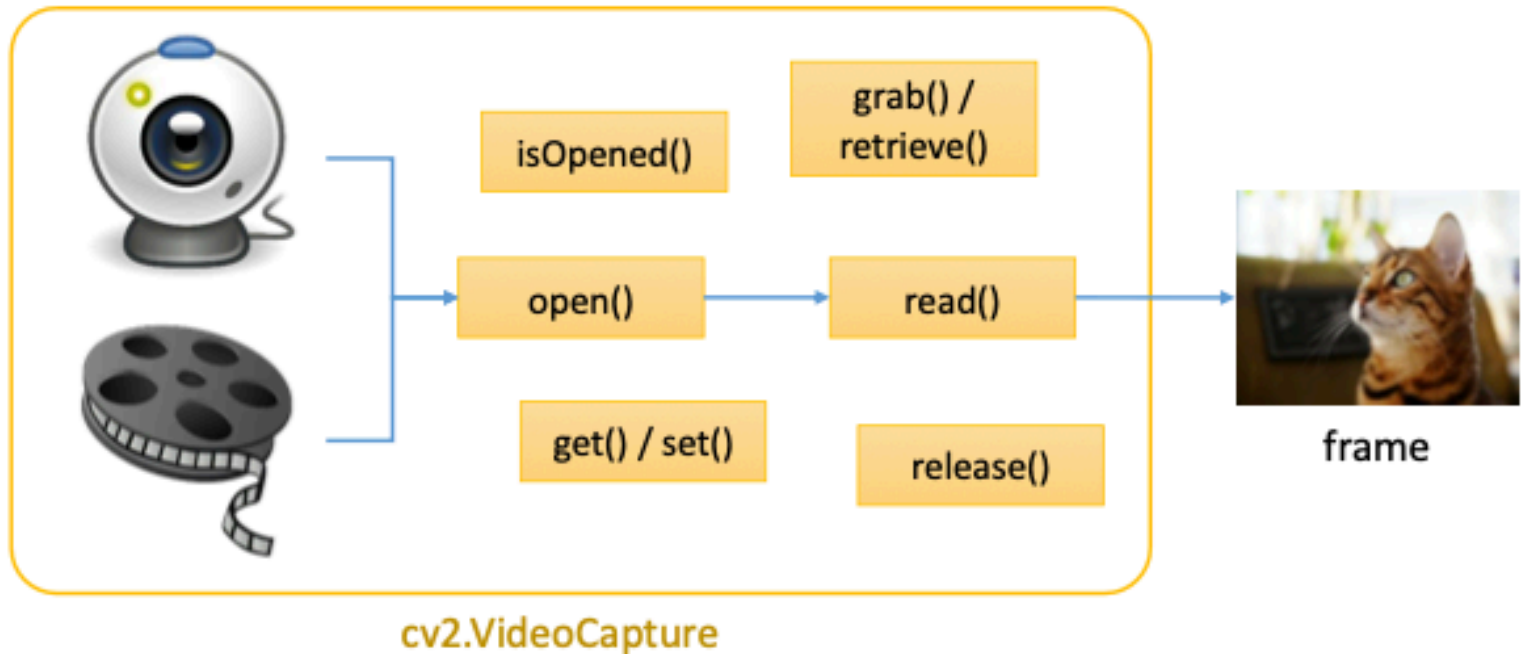
OpenCV-Python 기초 사용법

5) 카메라와 동영상 처리하기

카메라와 동영상 처리하기

■ cv2.VideoCapture 클래스

- OpenCV에서는 카메라와 동영상으로부터 프레임(frame)을 받아오는 작업을 cv2.VideoCapture 클래스 하나로 처리함



카메라와 동영상 처리하기

■ 카메라 열기

```
cv2.VideoCapture(index, apiPreference=None) -> retval
```

- index: camera_id + domain_offset_id
시스템 기본 카메라를 기본 방법으로 열려면 index에 0을 전달
- apiPreference: 선호하는 카메라 처리 방법을 지정
- retval: cv2.VideoCapture 객체

```
cv2.VideoCapture.open(index, apiPreference=None) -> retval
```

- retval: 성공하면 True, 실패하면 False.

카메라와 동영상 처리하기

■ 동영상, 정지 영상 시퀀스, 비디오 스트림 열기

```
cv2.VideoCapture(filename, apiPreference=None) -> retval
```

- filename: 비디오 파일 이름, 정지 영상 시퀀스, 비디오 스트림 URL 등
(e.g) 'video.avi', 'img_%02d.jpg', 'protocol://host:port/script?params|auth'
- apiPreference: 선호하는 동영상 처리 방법을 지정
- retval: cv2.VideoCapture 객체

```
cv2.VideoCapture.open(filename, apiPreference=None) -> retval
```

- retval: 성공하면 True, 실패하면 False.

카메라와 동영상 처리하기

■ 비디오 캡처가 준비되었는지 확인

```
cv2.VideoCapture.isOpened() -> retval
```

- retval: 성공하면 True, 실패하면 False.

■ 프레임 받아오기

```
cv2.VideoCapture.read(image=None) -> retval, image
```

- retval: 성공하면 True, 실패하면 False.
- image: 현재 프레임 (numpy.ndarray)

카메라와 동영상 처리하기

■ 카메라, 비디오 장치 속성 값 참조

```
cv2.VideoCapture.get(propId) -> retval
```

- propId: 속성 상수. ([OpenCV 문서](#) 참조)

CAP_PROP_FRAME_WIDTH	프레임 가로 크기
CAP_PROP_FRAME_HEIGHT	프레임 세로 크기
CAP_PROP_FPS	초당 프레임 수
CAP_PROP_FRAME_COUNT	비디오 파일의 총 프레임 수
CAP_PROP_POS_MSEC	밀리초 단위로 현재 위치
CAP_PROP_POS_FRAMES	현재 프레임 번호
CAP_PROP_EXPOSURE	노출

- retval: 성공하면 해당 속성 값, 실패하면 0.

카메라와 동영상 처리하기

■ 카메라, 비디오 장치 속성 값 참조

```
cv2.VideoCapture.set(propId, value) -> retval
```

- propId: 속성 상수
- value: 속성 값
- retval: 성공하면 True, 실패하면 False.

카메라와 동영상 처리하기

■ 카메라 처리 예제

```
import sys
import cv2
```

```
cap = cv2.VideoCapture(0)
```

기본 카메라 장치 열기.

```
while True:
```

```
    ret, frame = cap.read()
```

카메라로부터 프레임을 정상적으로 받아오면
ret에는 True, frame에는 해당 프레임이 저장됨.

```
    inversed = ~frame
```

현재 프레임 반전

```
    cv2.imshow('frame', frame)
```

```
    cv2.imshow('inversed', inversed)
```

```
    if cv2.waitKey(10) == 27:
        break
```

일정 시간(e.g. 10ms) 기다린 후 다음 프레임 처리.
만약 ESC 키를 누르면 while 루프 종료.

```
cap.release()
```

사용한 자원 해제

```
cv2.destroyAllWindows()
```

카메라와 동영상 처리하기

■ 동영상 처리 예제

```
cap = cv2.VideoCapture('video1.mp4')

fps = round(cap.get(cv2.CAP_PROP_FPS))
delay = round(1000 / fps)

while True:
    ret, frame = cap.read()

    inversed = ~frame

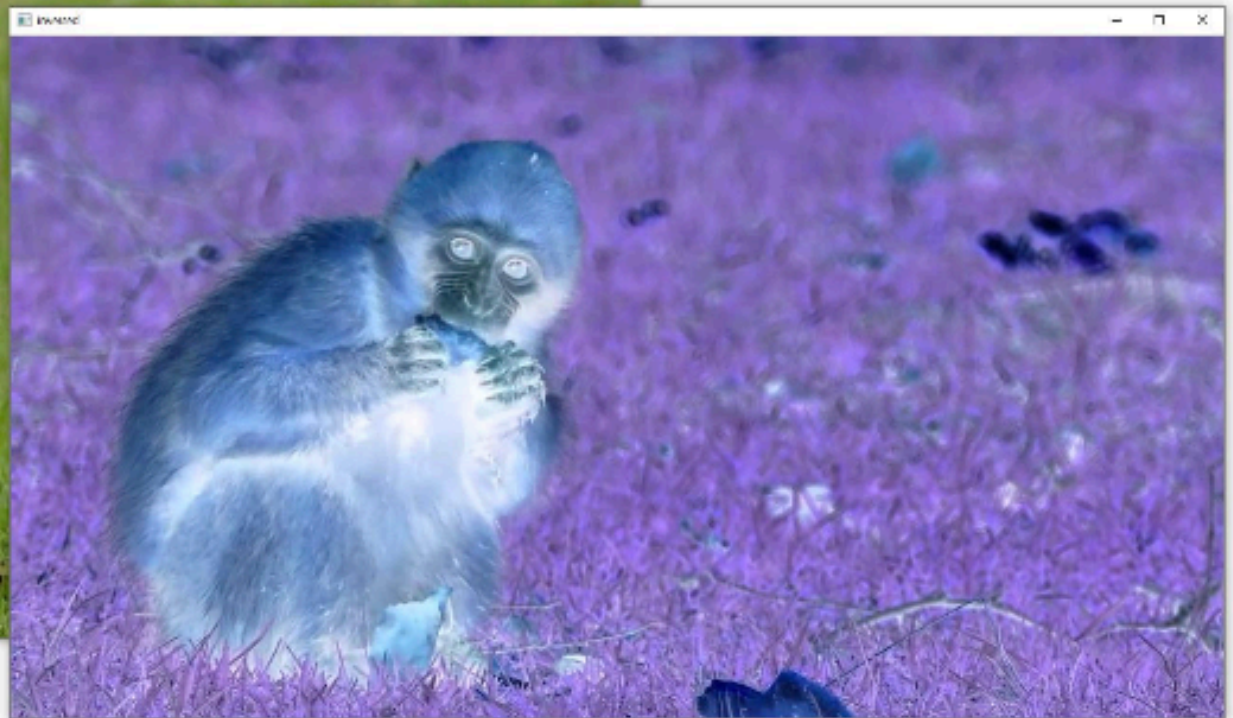
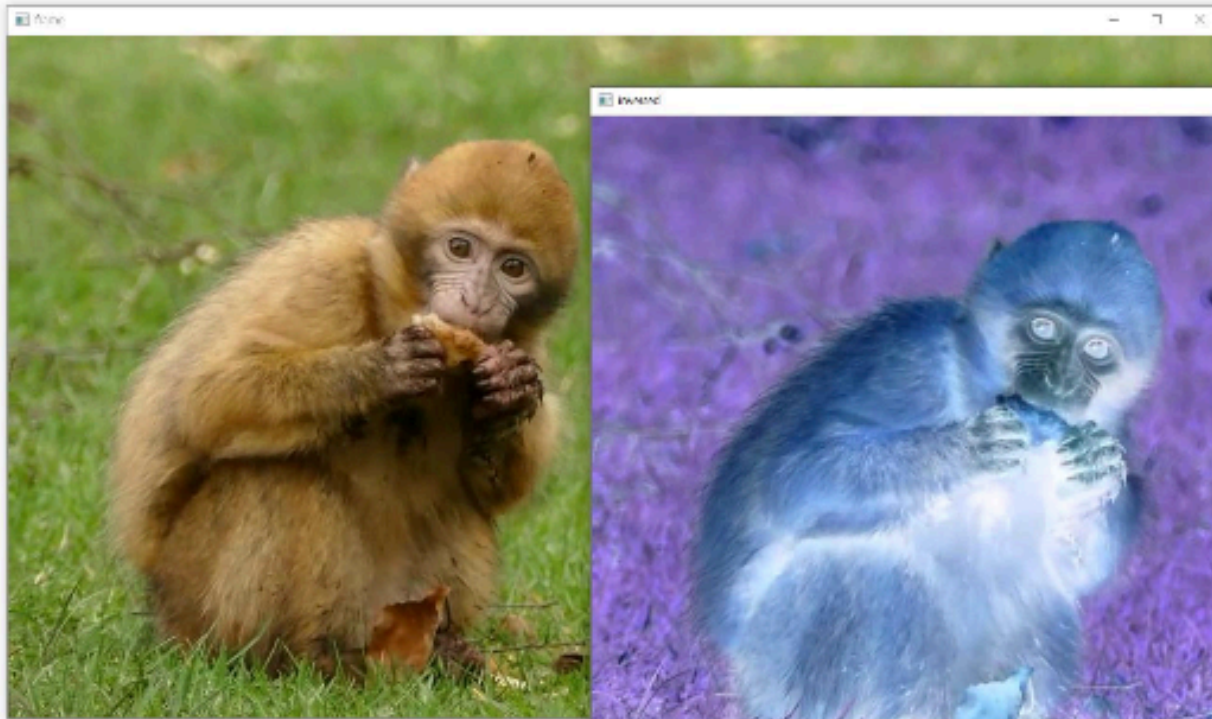
    cv2.imshow('frame', frame)
    cv2.imshow('inversed', inversed)

    if cv2.waitKey(delay) == 27:
        break

cap.release()
cv2.destroyAllWindows()
```

카메라와 동영상 처리하기

동영상 처리 예제 실행 결과



카메라와 동영상 처리하기

■ cv2.VideoWriter 클래스

- OpenCV에서는 cv2.VideoWriter 클래스를 이용하여 일련의 프레임을 동영상 파일로 저장할 수 있음
- 일련의 프레임은 모두 크기와 데이터 타입이 같아야 함

■ Fourcc (4-문자 코드, four character code)

- 동영상 파일의 코덱, 압축 방식, 색상, 픽셀 포맷 등을 정의하는 정수 값

cv2.VideoWriter_fourcc(*'DIVX')	DIVX MPEG-4 코덱
cv2.VideoWriter_fourcc(*'XVID')	XVID MPEG-4 코덱
cv2.VideoWriter_fourcc(*'FMP4')	FFMPEG MPEG-4 코덱
cv2.VideoWriter_fourcc(*'X264')	H.264/AVC 코덱
cv2.VideoWriter_fourcc(*'MJPG')	Motion-JPEG 코덱

<http://www.fourcc.org/codecs.php>

카메라와 동영상 처리하기

■ 저장을 위한 동영상 파일 열기

```
cv2.VideoWriter(filename, fourcc, fps, frameSize, isColor=None) -> retval
```

- filename: 비디오 파일 이름 (e.g. 'video.mp4')
- fourcc: fourcc (e.g. cv2.VideoWriter_fourcc(*'DIVX'))
- fps: 초당 프레임 수 (e.g. 30)
- frameSize: 프레임 크기. (width, height) 튜플.
- isColor: 컬러 영상이면 True, 그렇지 않으면 False.
- retval: cv2.VideoWriter 객체

```
cv2.VideoWriter.open(filename, fourcc, fps, frameSize, isColor=None) -> retval
```

- retval: 성공하면 True, 실패하면 False.

카메라와 동영상 처리하기

■ 비디오 파일이 준비되었는지 확인

```
cv2.VideoWriter.isOpened() -> retval
```

- retval: 성공하면 True, 실패하면 False.

■ 프레임 저장하기

```
cv2.VideoWriter.write(image) -> None
```

- image: 저장할 프레임 (numpy.ndarray)

카메라와 동영상 처리하기

■ 웹카메라 입력을 동영상으로 저장하기

```
cap = cv2.VideoCapture(0)

w = round(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = round(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fourcc = cv2.VideoWriter_fourcc(*'DIVX') # *'DIVX' == 'D','I','V','X'
out = cv2.VideoWriter('output.avi', fourcc, 30, (w, h))

while True:
    ret, frame = cap.read()

    inversed = ~frame
    out.write(inversed)

    cv2.imshow('frame', frame)
    cv2.imshow('inversed', inversed)
    if cv2.waitKey(10) == 27:
        break

...
```

카메라와 동영상 처리하기

```
import sys
import cv2
```

```
cap = cv2.VideoCapture(0)
```

```
if not cap.isOpened():
    print("Camera open failed!")
    sys.exit()
```

```
w = round(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
h = round(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = cap.get(cv2.CAP_PROP_FPS)
```

```
fourcc = cv2.VideoWriter_fourcc(*'DIVX') # '*'DIVX' == 'D', 'I', 'V', 'X'
delay = round(1000 / fps)
```

카메라와 동영상 처리하기

```
out = cv2.VideoWriter('output.avi', fourcc, fps, (w, h))

if not out.isOpened():
    print('File open failed!')
    cap.release()
    sys.exit()

while True:
    ret, frame = cap.read()

    if not ret:
        break

    inversed = ~frame

    out.write(inversed)

    cv2.imshow('frame', frame)
    cv2.imshow('inversed', inversed)

    if cv2.waitKey(delay) == 27:
        break

cap.release()
out.release()
cv2.destroyAllWindows()
```

OpenCV-Python 기초 사용법

6) 키보드 이벤트 처리하기

키보드 이벤트 처리하기

■ 키보드 입력 대기 함수

```
cv2.waitKey(delay=None) -> retval
```

- delay: 밀리초 단위 대기 시간. $\text{delay} \leq 0$ 이면 무한히 기다림. 기본값은 0.
- retval: 눌린 키 값(ASCII code). 키가 눌리지 않으면 -1.
- 참고 사항
 - cv2.waitKey() 함수는 OpenCV 창이 하나라도 있을 때 동작함
 - 특정 키 입력을 확인하려면 ord() 함수를 이용

```
while True:  
    if cv2.waitKey() == ord('q'):  
        break
```

- 주요 특수키 코드: 27(ESC), 13(ENTER), 9(TAB)

키보드 이벤트 처리하기

■ 키보드 특수키 입력 처리하기

- Windows 운영체제에서 방향키, 함수키 등의 특수키 입력은 `cv2.waitKeyEx()` 함수 사용

표 4-7 주요 특수 키에 해당하는 waitKeyEx() 함수 반환값

특수 키	waitKeyEx() 반환값	특수 키	waitKeyEx() 반환값
Insert	0x2d0000	F1	0x700000
Delete	0x2e0000	F2	0x710000
Home	0x240000	F3	0x720000
End	0x230000	F4	0x730000
Page Up	0x210000	F5	0x740000
Page Down	0x220000	F6	0x750000
←	0x250000	F7	0x760000
↑	0x260000	F8	0x770000
→	0x270000	F9	0x780000
↓	0x280000	F10	0x790000
		F11	0x7a0000
		F12	0x7b0000

키보드 이벤트 처리하기

- 키보드에서 'i' 또는 'I' 키를 누르면 영상을 반전

```
import cv2

img = cv2.imread('cat.bmp', cv2.IMREAD_GRAYSCALE)

cv2.imshow('image', img)

while True:
    keycode = cv2.waitKey()
    if keycode == ord('i') or keycode == ord('I'):
        img = ~img
        cv2.imshow('image', img)
    elif keycode == 27:
        break

cv2.destroyAllWindows()
```



OpenCV-Python 기초 사용법

7) 마우스 이벤트 처리하기

마우스 이벤트 처리하기

■ 마우스 이벤트 콜백함수 등록 함수

```
cv2.setMouseCallback(windowName, onMouse, param=None) -> None
```

- windowName: 마우스 이벤트를 처리할 창 이름
- onMouse: 마우스 이벤트를 처리할 콜백 함수 이름.
마우스 이벤트 콜백 함수는 다음 형식을 따라야 함.

```
onMouse(event, x, y, flags, param) -> None
```

- param: 콜백 함수에 전달할 데이터

마우스 이벤트 처리하기

■ 마우스 이벤트 처리 함수(콜백 함수) 형식

```
onMouse(event, x, y, flags, param) -> None
```

- event: 마우스 이벤트 종류. cv2.EVENT_로 시작하는 상수.
- x: 마우스 이벤트가 발생한 x 좌표
- y: 마우스 이벤트가 발생한 y 좌표
- flags: 마우스 이벤트 발생 시 상태. cv2.EVENT_FLAG_로 시작하는 상수
- param: cv2.setMouseCallback() 함수에서 설정한 데이터.

마우스 이벤트 처리하기

■ 마우스 이벤트 처리 함수의 event 인자

MouseEventTypes 열거형 상수	값	설명
cv2.EVENT_MOUSEMOVE	0	마우스가 창 위에서 움직이는 경우
cv2.EVENT_LBUTTONDOWN	1	마우스 왼쪽 버튼이 눌러지는 경우
cv2.EVENT_RBUTTONDOWN	2	마우스 오른쪽 버튼이 눌러지는 경우
cv2.EVENT_MBUTTONDOWN	3	마우스 가운데 버튼이 눌러지는 경우
cv2.EVENT_LBUTTONUP	4	마우스 왼쪽 버튼이 떼어지는 경우
cv2.EVENT_RBUTTONUP	5	마우스 오른쪽 버튼이 떼어지는 경우
cv2.EVENT_MBUTTONUP	6	마우스 가운데 버튼이 떼어지는 경우
cv2.EVENT_LBUTTONDBLCLK	7	마우스 왼쪽 버튼을 더블클릭하는 경우
cv2.EVENT_RBUTTONDBLCLK	8	마우스 오른쪽 버튼을 더블클릭하는 경우
cv2.EVENT_MBUTTONDBLCLK	9	마우스 가운데 버튼을 더블클릭하는 경우
cv2.EVENT_MOUSEWHEEL	10	마우스 휠을 앞뒤로 돌리는 경우
cv2.EVENT_MOUSEHWHEEL	11	마우스 휠을 좌우로 움직이는 경우

마우스 이벤트 처리하기

■ 마우스 이벤트 처리 함수의 flags 인자

MouseEventFlags 열거형 상수	값	설명
cv2.EVENT_FLAG_LBUTTON	1	마우스 왼쪽 버튼이 눌러져 있음
cv2.EVENT_FLAG_RBUTTON	2	마우스 오른쪽 버튼이 눌러져 있음
cv2.EVENT_FLAG_MBUTTON	4	마우스 가운데 버튼이 눌러져 있음
cv2.EVENT_FLAG_CTRLKEY	8	CTRL 키가 눌러져 있음
cv2.EVENT_FLAG_SHIFTKEY	16	SHIFT 키가 눌러져 있음
cv2.EVENT_FLAG_ALTKEY	32	ALT 키가 눌러져 있음

마우스 이벤트 처리하기

■ 마우스를 이용한 그리기 예제

```
oldx = oldy = -1

def on_mouse(event, x, y, flags, param):
    global oldx, oldy

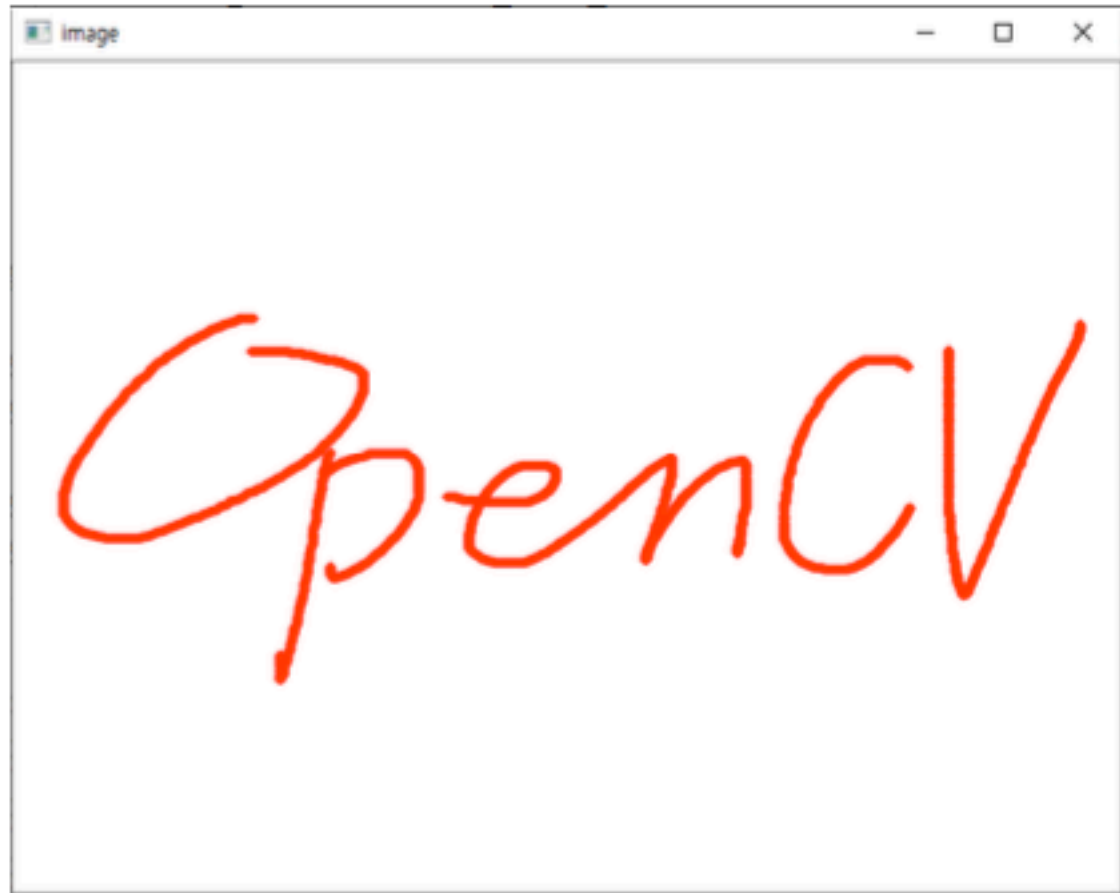
    if event == cv2.EVENT_LBUTTONDOWN:
        oldx, oldy = x, y
    elif event == cv2.EVENT_MOUSEMOVE:
        if flags & cv2.EVENT_FLAG_LBUTTON:
            cv2.line(img, (oldx, oldy), (x, y), (0, 0, 255), 4, cv2.LINE_AA)
            cv2.imshow('image', img)
            oldx, oldy = x, y

img = np.ones((480, 640, 3), dtype=np.uint8) * 255

cv2.imshow('image', img)
cv2.setMouseCallback('image', on_mouse)
cv2.waitKey()
```

마우스 이벤트 처리하기

- 마우스를 이용한 그리기 예제



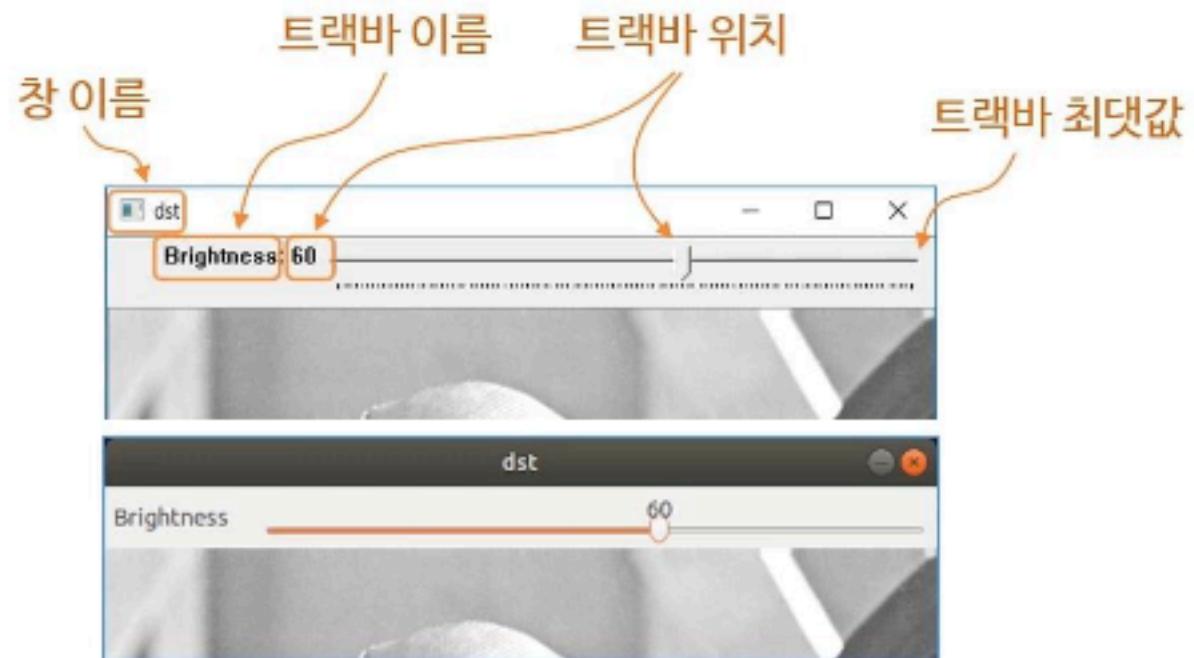
OpenCV-Python 기초 사용법

8) 트랙바 사용하기

트랙바 사용하기

■ 트랙바(Trackbar)란?

- 프로그램 동작 중 사용자가 지정한 범위 안의 값을 선택할 수 있는 컨트롤
- OpenCV에서 제공하는 (유일한?) 그래픽 사용자 인터페이스



트랙바 사용하기

■ 트랙바 생성 함수

```
cv2.createTrackbar(trackbarName, windowName, value, count, onChange) -> None
```

- trackbarName: 트랙바 이름
- windowName: 트랙바를 생성할 창 이름.
- value: 트랙바 위치 초기값
- count: 트랙바 최댓값. 최솟값은 항상 0.
- onChange: 트랙바 위치가 변경될 때마다 호출할 콜백 함수 이름

트랙바 이벤트 콜백 함수는 다음 형식을 따름.

```
onChange(pos) -> None
```

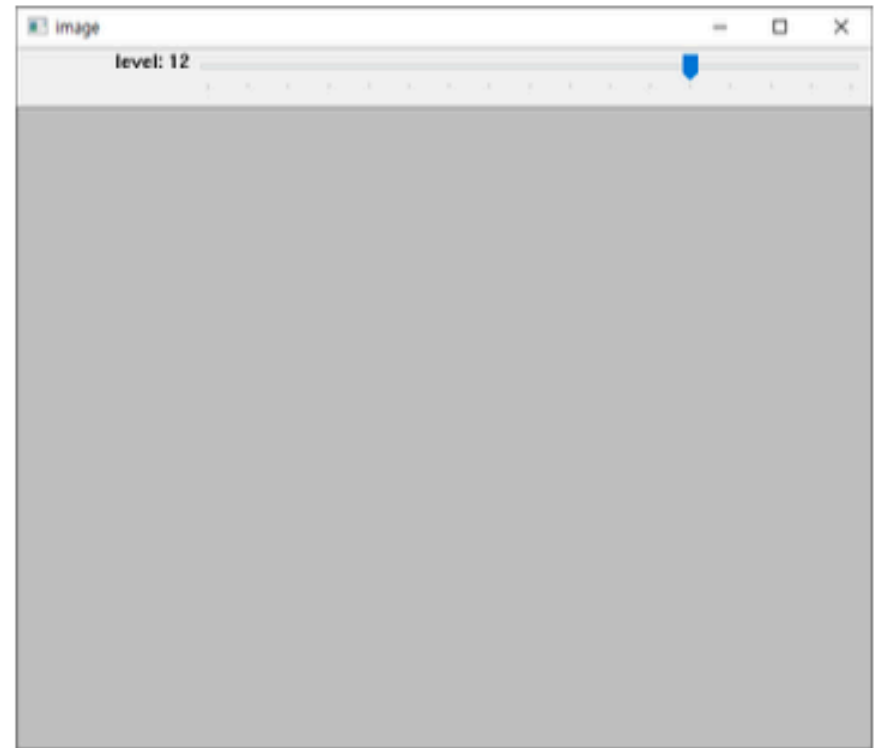
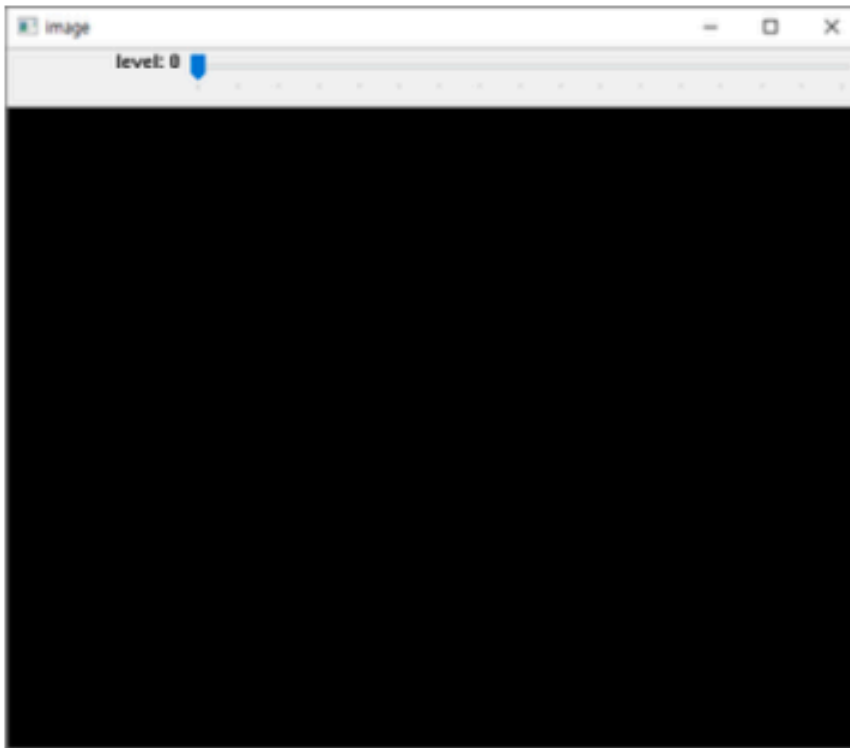
트랙바 사용하기

- 트랙바를 이용한 그레이스케일 레벨 표현

```
def on_level_change(pos):  
    value = pos * 16  
    if value >= 255:  
        value = 255  
  
    img[:] = value  
    cv2.imshow('image', img)  
  
img = np.zeros((480, 640), np.uint8)  
  
cv2.namedWindow('image')  
cv2.createTrackbar('level', 'image', 0, 16, on_level_change)  
  
cv2.imshow('image', img)  
cv2.waitKey()  
cv2.destroyAllWindows()
```

트랙바 사용하기

- 트랙바를 이용한 그레이스케일 레벨 표현



OpenCV-Python 기초 사용법

9) 연산 시간 측정 방법

연산 시간 측정 방법

- OpenCV에서는 TickMeter 클래스를 이용하여 연산 시간을 측정

```
cv2.TickMeter() -> tm
```

- tm: cv2.TickMeter 객체
- tm.start(): 시간 측정 시작
- tm.stop(): 시간 측정 끝
- tm.reset(): 시간 측정 초기화
- tm.getTimeSec(): 측정 시간을 초 단위로 반환
- tm.getTimeMilli(): 측정 시간을 밀리 초 단위로 반환
- tm.getTimeMicro(): 측정 시간을 마이크로 초 단위로 반환

연산 시간 측정 방법

■ 특정 연산의 시간 측정 예제

```
img = cv2.imread('hongkong.jpg')

tm = cv2.TickMeter()
tm.start()

edge = cv2.Canny(img, 50, 150)

tm.stop()

print('Elapsed time: {}ms.'.format(tm.getTimeMilli()))
```

OpenCV-Python 기초 사용법

10) 실전 코딩 : 동영상 전환 이펙트

[실전 코딩] 동영상 전환 이펙트

■ 동영상 전환 이펙트

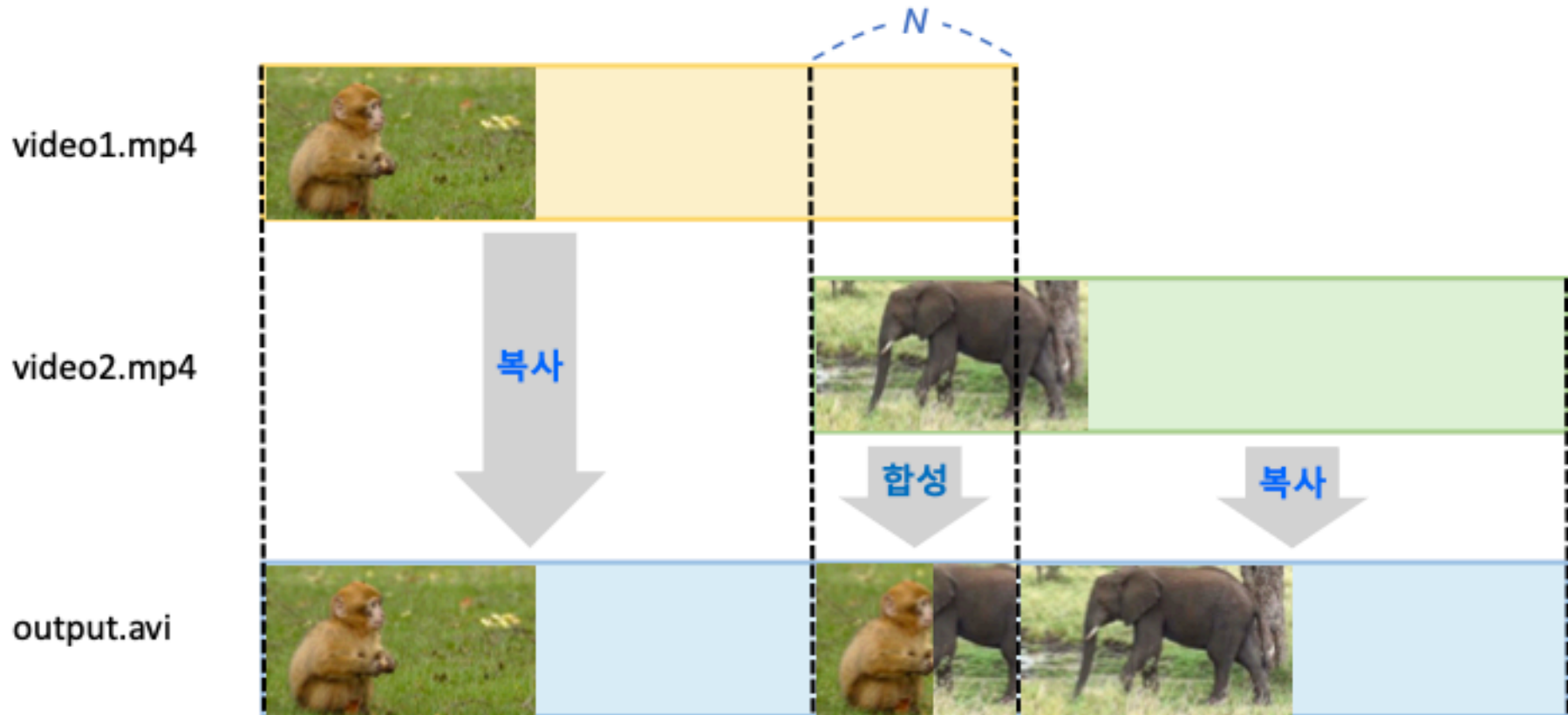
- 두 동영상 클립 사이에 추가되는 애니메이션 효과
- 페이드-인(fade-in), 페이드-아웃(fade-out), 디졸브(dissolve), 밀기, 확대 등

■ 구현 할 기능

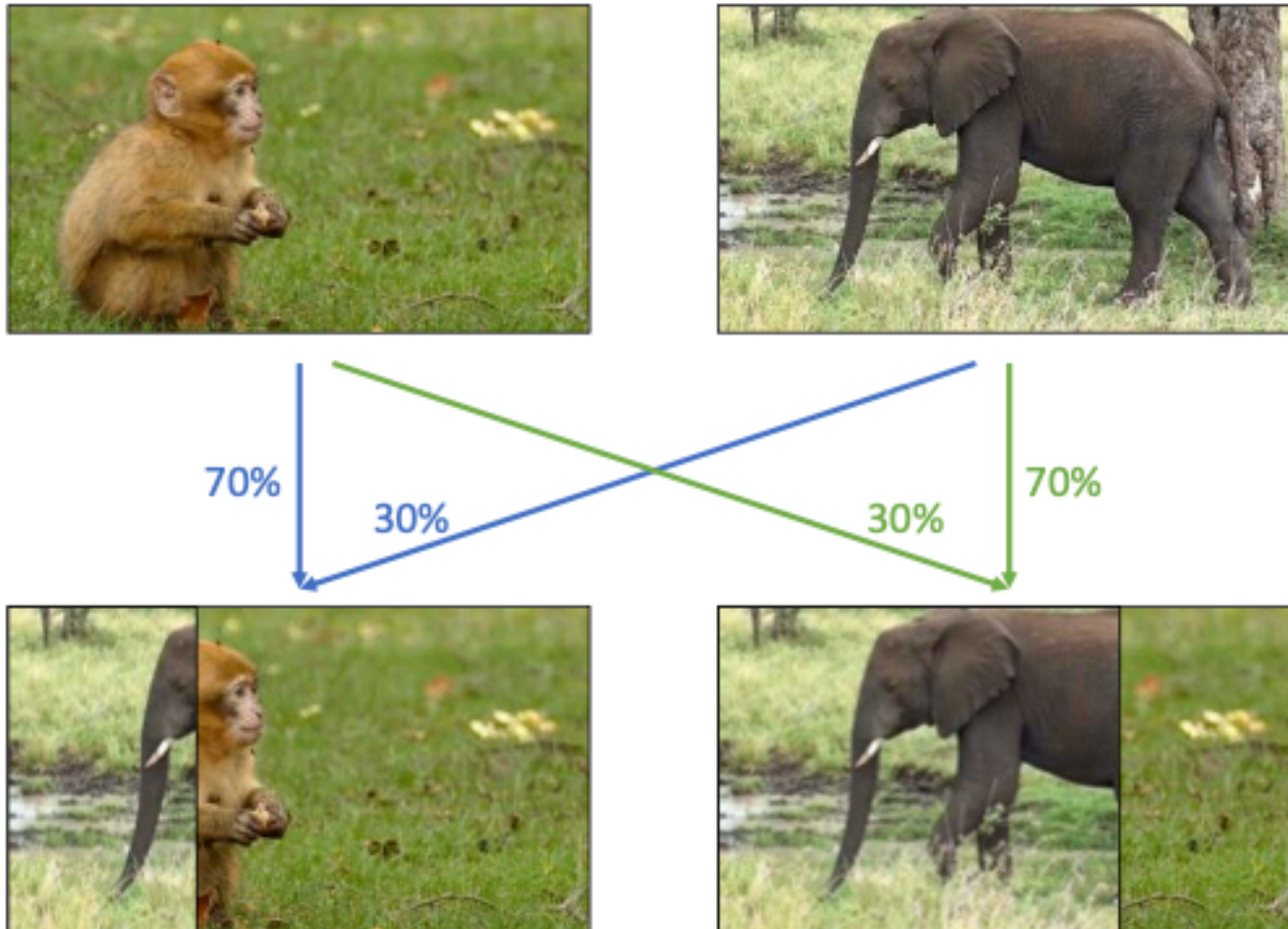
- 두 개의 동영상 동시 열기
- 첫 번째 동영상의 마지막 N개 프레임과 두 번째 동영상의 처음 N개 프레임을 합성
- 합성된 영상을 동영상으로 저장하기

[실전 코딩] 동영상 전환 이펙트

```
effect_frames = int(fps * 2)
```



[실전 코딩] 동영상 전환 이펙트



[실전 코딩] 동영상 전환 이펙트

```
import sys
import numpy as np
import cv2

# 두 개의 동영상을 열어서 cap1, cap2로 지정
cap1 = cv2.VideoCapture("video1.mp4")
cap2 = cv2.VideoCapture("video2.mp4")

if not cap1.isOpened() or not cap2.isOpened():
    print("video open failed!")
    sys.exit()

# 두 동영상의 크기, FPS는 같다고 가정함
frame_cnt1 = round(cap1.get(cv2.CAP_PROP_FRAME_COUNT))
frame_cnt2 = round(cap2.get(cv2.CAP_PROP_FRAME_COUNT))
fps = cap1.get(cv2.CAP_PROP_FPS)
effect_frames = int(fps * 2)

print("frame_cnt1:", frame_cnt1)
print("frame_cnt2:", frame_cnt2)
print("FPS:", fps)

fps2 = cap2.get(cv2.CAP_PROP_FPS)
print("fps2:", fps2)

delay = int(1000 / fps)
print("delay:", delay)
```

[실전 코딩] 동영상 전환 이펙트

```
w = round(cap1.get(cv2.CAP_PROP_FRAME_WIDTH))
h = round(cap1.get(cv2.CAP_PROP_FRAME_HEIGHT))
fourcc = cv2.VideoWriter_fourcc(*"DIVX")

# 출력 동영상 객체 생성
out = cv2.VideoWriter("output.avi", fourcc, fps, (w, h))

# 1번 동영상 복사
for i in range(frame_cnt1 - effect_frames):
    ret1, frame1 = cap1.read()

    if not ret1:
        print("frame read error!")
        sys.exit()

    out.write(frame1)
    print(".", end="")

cv2.imshow("output", frame1)
cv2.waitKey(delay)
```

[실전 코딩] 동영상 전환 이펙트

```
# 1번 동영상 뒷부분과 2번 동영상 앞부분을 합성
for i in range(effect_frames):
    ret1, frame1 = cap1.read()
    ret2, frame2 = cap2.read()

    if not ret1 or not ret2:
        print("frame read error!")
        sys.exit()

    dx = int(w / effect_frames) * i

    frame = np.zeros((h, w, 3), dtype=np.uint8)
    frame[:, 0:dx, :] = frame2[:, 0:dx, :]
    frame[:, dx:w, :] = frame1[:, dx:w, :]

    # alpha = i / effect_frames
    # frame = cv2.addWeighted(frame1, 1 - alpha, frame2, alpha, 0)

    out.write(frame)
    print(".", end="")

cv2.imshow("output", frame)
cv2.waitKey(delay)
```


[실전 코딩] 동영상 전환 이펙트

```
# 2번 동영상을 복사
for i in range(effect_frames, frame_cnt2):
    ret2, frame2 = cap2.read()

    if not ret2:
        print("frame read error!")
        sys.exit()

    out.write(frame2)
    print(".", end="")

    cv2.imshow("output", frame2)
    cv2.waitKey(delay)

print("\noutput.avi file is successfully generated!")

cap1.release()
cap2.release()
out.release()
cv2.destroyAllWindows()
```

[실전 코딩] 동영상 전환 이펙트

[실습] 동영상 다른 전환 이펙트 구현

검색, chatGPT, Bard 가능